

Lab worksheet — control flow, slices, maps

Name: ___ · GitHub: ___

Fill this **before** you leave. Then tick the [lab sheet](#).

A. Control flow

Question	Your answer
Does <code>if score {</code> compile when <code>score</code> is <code>int</code> ?	
Are braces required on a one-line <code>if</code> ?	
What is a tagless <code>switch</code> ?	
Go's only loop keyword is...	

B. Slice gotcha

Run (or predict, then run):

```
a := []int{10, 20, 30, 40}
b := a[:2]
b = append(b, 99)
fmt.Println(a, b, cap(b))
```

Prompt	Write it
<code>cap(b)</code> before append	
<code>a</code> after append	
One sentence: why did <code>a</code> change (or not)?	

C. Maps / comma-ok

Code	What prints / happens?
<code>var m map[string]int; m["x"] = 1</code>	
<code>m := make(map[string]int); fmt.Println(m["x"])</code>	

Code	What prints / happens?
<code>v, ok := m["x"]</code> when missing	

D. Paper test table for `contains`

Fill **before** you code.

list	want	result
<code>["SE", "CS"]</code>	<code>"SE"</code>	
<code>["SE", "CS"]</code>	<code>"IT"</code>	
<code>[]</code>	<code>"SE"</code>	
<code>["SE", "SE"]</code>	<code>"SE"</code>	

E. Labs (tick when done)

- ☐ `Contains` tests green
- ☐ `Unique` preserves first-seen order; tests green
- ☐ `Freq` tests green (comma-ok assertions)
- ☐ I can explain the backing-array gotcha in one sentence